



Cube Runner Arena

Clase práctica de IA aplicada a videojuegos con
Unity 3D y C#

Clase 2: prototipado guiado con ChatGPT gratuito



Qué haremos hoy

- Construir un minijuego completo con primitivas básicas de Unity.
- Usar ChatGPT gratuito como asistente técnico, no como autoridad.
- Practicar prompts para código, configuración, revisión y depuración.
- Cerrar el ciclo: escena, scripts, UI, prueba y entrega.



La regla de la clase

ChatGPT no decide por nosotros.

Acelera el trabajo cuando el problema está bien definido y la respuesta se comprueba dentro de Unity.

Pedir

Contexto, objetivo y restricciones.

Probar

Play Mode, consola e Inspector.

Corregir

Volver con datos, no con intuiciones.



Ruta de 5 horas

Apertura
20 min

Escena base
30 min

Player
40 min

Coleccionables
40 min

Estados y UI
45 min

Obstáculos
45 min

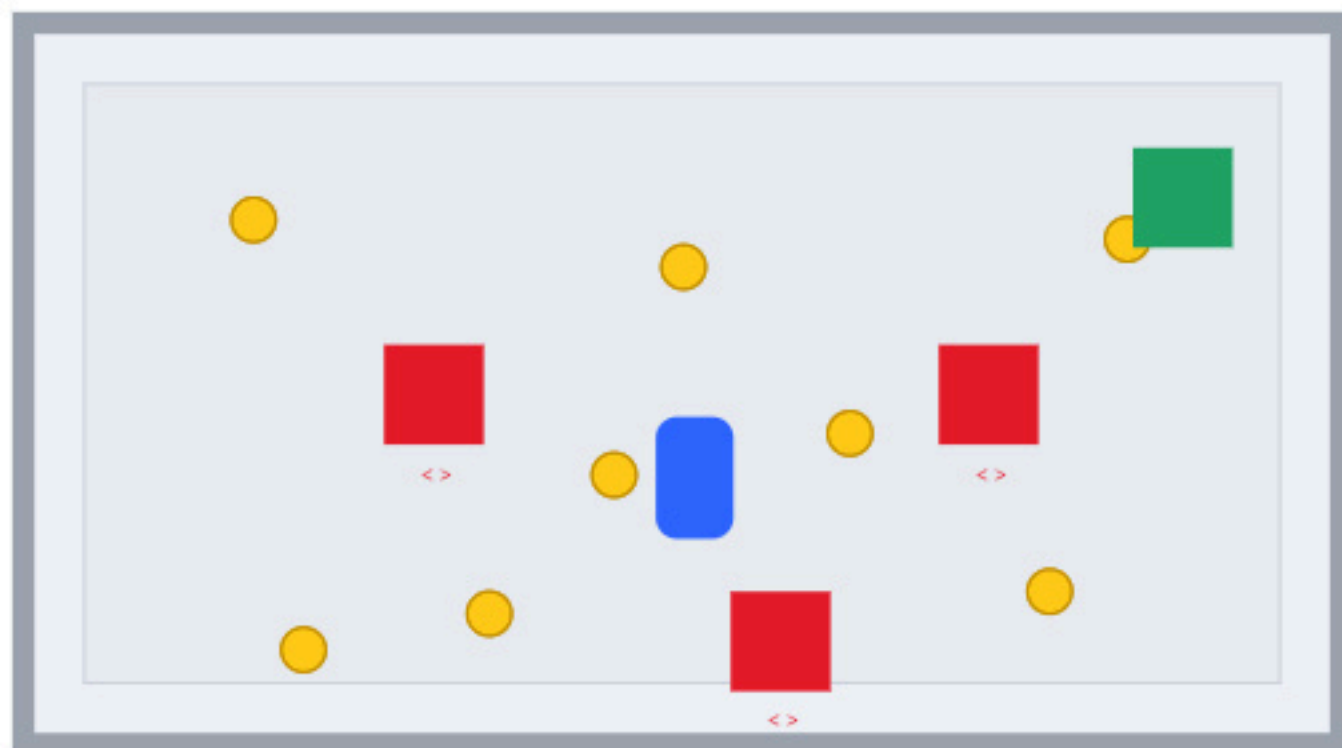
Cámara y pulido
35 min

Debugging IA
30 min

Entrega
15 min



Minijuego objetivo



- Cápsula azul: jugador.
- Esferas amarillas: coleccionables.
- Cubos rojos: obstáculos móviles.
- Cubos grises: paredes.
- Cubo verde: objetivo visual.

Ganar: recoger todo. Perder: tocar obstáculo o agotar tiempo.



Prompt inicial de la práctica

Actúa como desarrollador Unity C# senior.
Estoy creando Cube Runner Arena con primitivas básicas:
cápsula, cubos, esferas y plano.

Restricciones:

- Unity 3D y C#
- ChatGPT gratuito, sin suscripción
- no assets externos, DOTween ni Cinemachine
- no nuevo Input System salvo que lo pida
- cambios mínimos, scripts claros
- indica qué revisar en Inspector

Antes de código: lista GameObjects, componentes y riesgos.



Escena base



Player

Cápsula + Rigidbody + tag Player.

Collectibles

Esferas con Collider en Is Trigger.

Obstacles

Cubos rojos con Collider normal.

Canvas

ScoreText, TimeText y MessageText.

Antes del primer script, la escena ya debe contar la mecánica.



El primer prompt no pide código

Estoy preparando una escena básica de Unity 3D para Cube Runner Arena.
Uso solo plano, cápsula, esfera y cubo.

Dame una checklist corta para:

- player con Rigidbody
- suelo con Collider
- paredes
- cámara
- materiales
- jerarquía

No generes código todavía.

Objetivo

Reducir incertidumbre antes de programar.

Resultado

Lista de objetos y componentes.

Verificación

Todo se comprueba en la escena.



Movimiento del jugador

Input

WASD o flechas.
Se lee en Update.

Física

Rigidbody.MovePosition.
Se aplica en FixedUpdate.

Inspector

Move Speed editable.
Congelar rotación X/Z.

Prompt:

Necesito PlayerMovement para Unity 3D.
Player = cápsula con Rigidbody.
Movimiento en plano XZ.
Usa Rigidbody.MovePosition en FixedUpdate.
No uses el nuevo Input System.
Velocidad con SerializeField.
Código completo + Inspector.



PlayerMovement.cs

```
using UnityEngine;

[RequireComponent(typeof(Rigidbody))]
public class PlayerMovement : MonoBehaviour
{
    [SerializeField] private float moveSpeed = 6f;
    private Rigidbody rb;
    private Vector3 moveDirection;

    private void Awake() => rb = GetComponent<Rigidbody>();

    private void Update()
    {
        float h = Input.GetAxisRaw("Horizontal");
        float v = Input.GetAxisRaw("Vertical");
        moveDirection = new Vector3(h, 0f, v).normalized;
    }

    private void FixedUpdate()
    {
        if (rb == null) return;
        rb.MovePosition(rb.position + moveDirection * moveSpeed * Time.fixedDeltaTime);
    }
}
```



Error frecuente: Input System

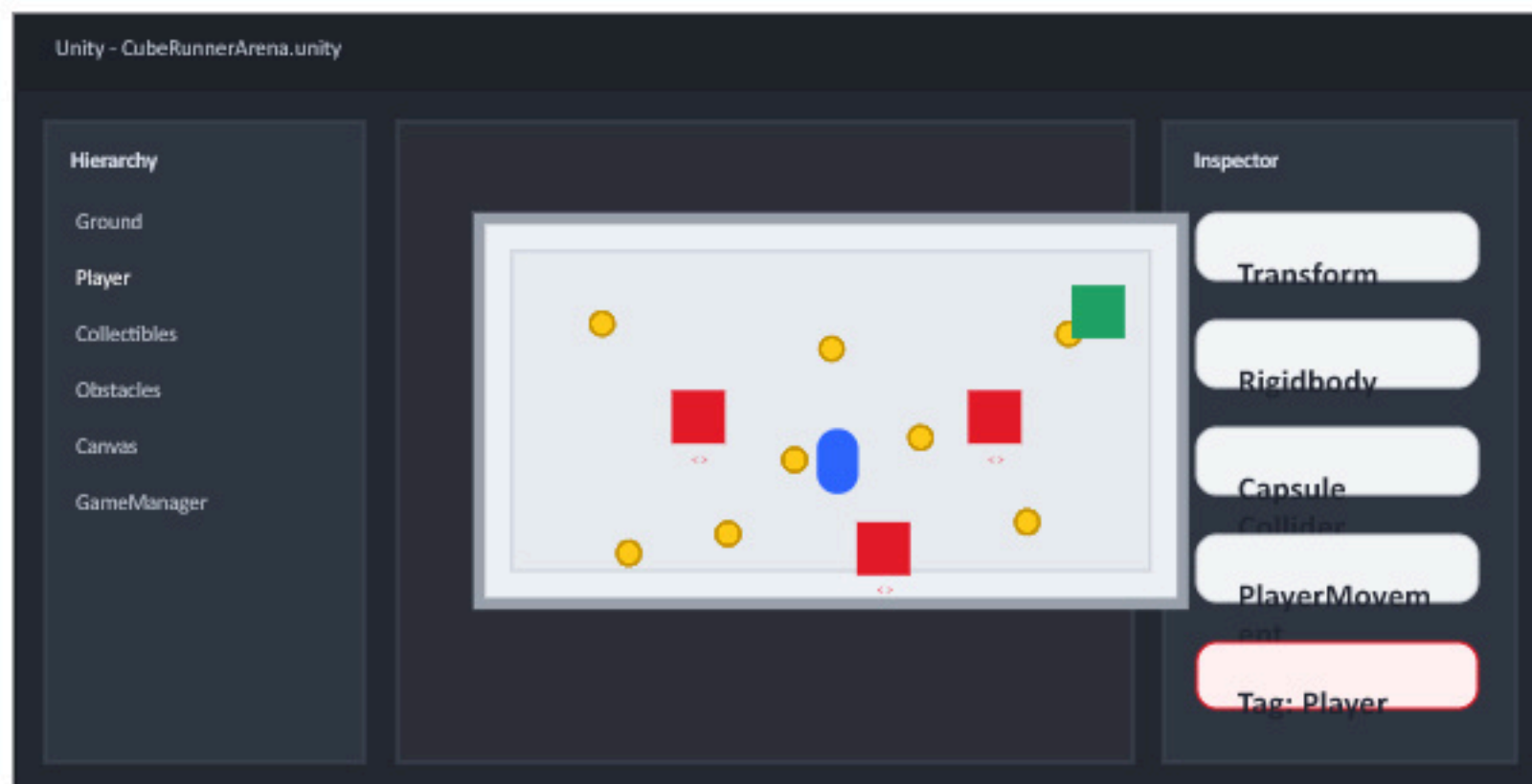
```
InvalidOperationException:  
You are trying to read Input using the UnityEngine.Input class,  
but active Input handling is Input System package.
```

Para esta práctica: Active Input Handling = Both

- Edit > Project Settings > Player > Other Settings.
- Cambiar a Both para permitir Input.GetAxisRaw.
- Reiniciar Unity si lo pide.



Coleccionables y score



Sphere

Collider marcado como Is Trigger.

Player

Tag Player y Rigidbody.

GameManager

ScoreText asignado en Inspector.

Si el trigger no salta, casi siempre falta una pieza de Inspector.



Collectible.cs

```
using UnityEngine;

public class Collectible : MonoBehaviour
{
    [SerializeField] private int points = 1;
    private bool collected;

    private void OnTriggerEnter(Collider other)
    {
        if (collected) return;
        if (!other.CompareTag("Player")) return;

        collected = true;

        GameManager gameManager = FindObjectOfType<GameManager>();
        if (gameManager != null) gameManager.AddScore(points);

        gameObject.SetActive(false);
    }
}
```




GameManager: primera versión

Responsabilidades:

- guardar score
- contar coleccionables totales al empezar
- actualizar ScoreText
- mostrar victoria al recoger todo

Restricciones:

- no eventos todavía
- no ScriptableObjects
- null-checks simples para UI
- TextMeshProUGUI asignado en Inspector

```
public void AddScore(int points)
{
    if (gameEnded) return;

    score += points;
    collectedCount++;

    UpdateScoreUI();

    if (collectedCount >= totalCollectibles)
        WinGame();
}
```

El manager coordina el prototipo; no convierte todo en sistema.



Estados, tiempo y reinicio

Inicio

Start() configura escena

Jugando

Update() descuenta tiempo

Victoria

todos los coleccionables

Derrota

tiempo 0 u obstáculo

Bloqueo

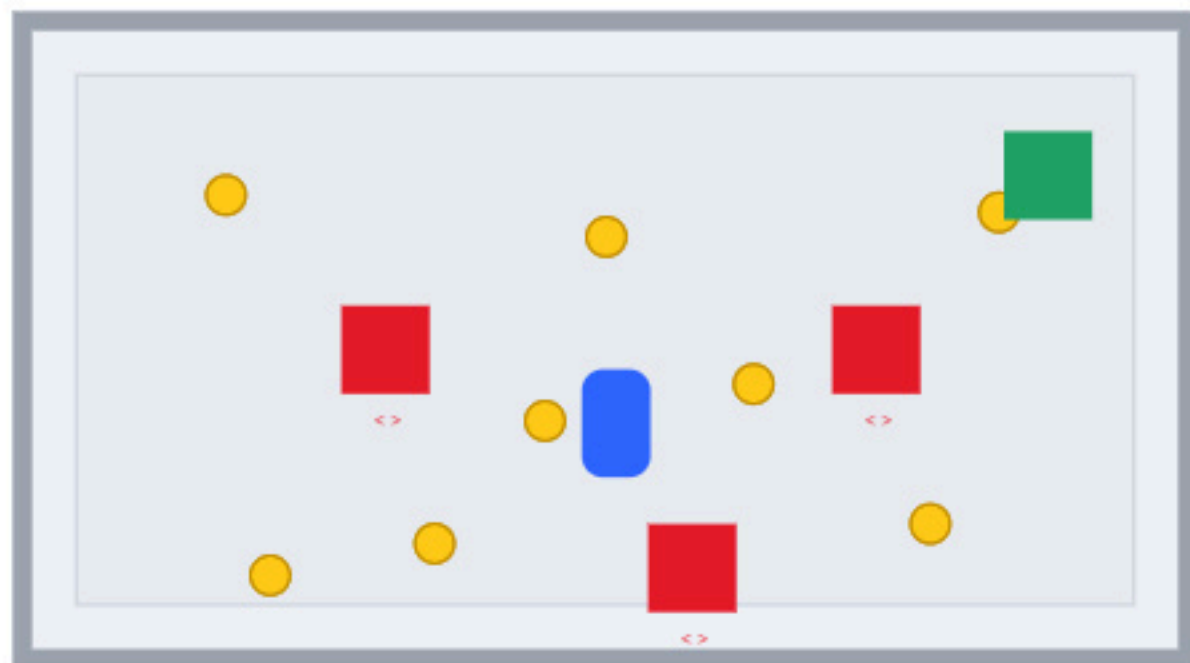
gameEnded evita dobles eventos

Reinicio

tecla R + LoadScene

No usamos Time.timeScale en esta práctica: mantenemos el flujo simple y visible.

Obstáculos con cubos rojos



- Movimiento entre dos puntos.
- `Vector3.MoveTowards`.
- Collider normal, no trigger.
- `OnCollisionEnter` llama a `LoseGame()`.
- La dificultad se ajusta con distancia y velocidad.



MovingObstacle.cs

```
public class MovingObstacle : MonoBehaviour
{
    [SerializeField] private float moveDistance = 4f;
    [SerializeField] private float moveSpeed = 3f;
    [SerializeField] private Vector3 moveDirection = Vector3.right;

    private Vector3 targetA, targetB, currentTarget;
    private GameManager gameManager;

    private void Start()
    {
        Vector3 dir = moveDirection.normalized;
        targetA = transform.position - dir * moveDistance;
        targetB = transform.position + dir * moveDistance;
        currentTarget = targetB;
        gameManager = FindObjectOfType<GameManager>();
    }
}
```

En clase se completa con Update() y OnCollisionEnter().



Cámara y pulido

- Opción mínima: cámara fija inclinada.
- Opción práctica: CameraFollow con offset.
- Usar LateUpdate para evitar vibración.
- No usar Cinemachine: menos dependencias para una clase inicial.

```
Prompt:  
Necesito CameraFollow simple.  
Target = Player.  
Mantener offset configurable.  
Usar LateUpdate.  
No usar Cinemachine.  
Código completo + Inspector.
```



Unity no se arregla solo con código



El error

NullReferenceException en GameManager.

Lo que mira ChatGPT

Script, consola y contexto.

Lo que mira Unity

Inspector, escena, tags y componentes.

Regla práctica: script + captura + consola + qué hay en Inspector.



Prompt de depuración

Tengo este error en Unity:
[pegar error completo]

Este es el script:
[pegar script]

Contexto de la escena:

- GameObject donde está el script:
- componentes del Player:
- tag del Player:
- colliders:
- Rigidbody:
- referencias asignadas en Inspector:

Dime:

1. causa más probable
2. cómo comprobarlo en Unity
3. cambio mínimo
4. qué no debería tocar



Bugs típicos de la práctica

Input no funciona

Active Input Handling no está en Both.

Trigger no salta

Is Trigger, tag Player o Rigidbody.

Colisión no salta

Algún collider está en trigger.

Texto no cambia

Referencia UI no asignada.

Cámara vibra

Seguir en Update en vez de LateUpdate.

LoadScene falla

Escena no guardada o Build Settings.



Alucinaciones en Unity

- APIs inventadas o paquetes no instalados.
- Mezcla de Rigidbody y Rigidbody2D.
- Input System nuevo cuando el proyecto no lo usa.
- Managers innecesarios para una mecánica simple.
- Errores con tags, colliders, escenas o Time.timeScale.



Antes de aceptar código generado

Compila sin errores.

Usa Unity 3D, no una solución 2D.

Respetar Inspector, tags y escena.

No introduce sistemas innecesarios.

No cambia nombres públicos sin motivo.

Se prueba en Play Mode.



Actividad final por grupos

- Terminar una escena jugable.
- Mostrar un prompt útil y un prompt que falló.
- Explicar una corrección de Inspector.
- Hacer una prueba completa en Play Mode.

Pregunta clave

¿Qué parte era código y qué parte era configuración de Unity?



Checklist de entrega

- Player se mueve y no gira raro.
- Score aumenta al recoger esferas.
- El tiempo baja y puede provocar derrota.
- Obstáculos causan Game Over.
- Victoria al recoger todo.
- Tecla R reinicia la escena.
- Textos UI asignados.
- Tag Player correcto.
- Colliders y Rigidbody revisados.
- Escena guardada.
- Build Settings revisado si hace falta.
- No hay errores en consola.



Scripts finales

PlayerMovement.cs

Input + Rigidbody.MovePosition.

Collectible.cs

Trigger + score + desactivar.

GameManager.cs

Score, tiempo, estado y reinicio.

MovingObstacle.cs

Patrulla + derrota por colisión.

CameraFollow.cs

Seguimiento simple con offset.

Cinco scripts. Ningún sistema nuevo innecesario.



Cierre

La IA acelera el prototipo, no sustituye la prueba.

El flujo profesional es: describir bien el problema, pedir ayuda concreta, aplicar cambios pequeños, probar en Unity, volver con contexto y verificar antes de confiar.



Anexo: PlayerMovement completo

```
using UnityEngine;

[RequireComponent(typeof(Rigidbody))]
public class PlayerMovement : MonoBehaviour
{
    [SerializeField] private float moveSpeed = 6f;
    private Rigidbody rb;
    private Vector3 moveDirection;

    private void Awake() => rb = GetComponent<Rigidbody>();

    private void Update()
    {
        float horizontal = Input.GetAxisRaw("Horizontal");
        float vertical = Input.GetAxisRaw("Vertical");
        moveDirection = new Vector3(horizontal, 0f, vertical).normalized;
    }

    private void FixedUpdate()
    {
        if (rb == null) return;
        Vector3 nextPosition = rb.position + moveDirection * moveSpeed * Time.fixedDeltaTime;
        rb.MovePosition(nextPosition);
    }
}
```



Anexo: Collectible completo

```
using UnityEngine;

public class Collectible : MonoBehaviour
{
    [SerializeField] private int points = 1;
    private bool collected;

    private void OnTriggerEnter(Collider other)
    {
        if (collected) return;
        if (!other.CompareTag("Player")) return;

        collected = true;

        GameManager gameManager = FindObjectOfType<GameManager>();
        if (gameManager != null)
            gameManager.AddScore(points);

        gameObject.SetActive(false);
    }
}
```



Anexo: GameManager 1/3

```
using TMPro;
using UnityEngine;
using UnityEngine.SceneManagement;

public class GameManager : MonoBehaviour
{
    [SerializeField] private TextMeshProUGUI scoreText;
    [SerializeField] private TextMeshProUGUI timeText;
    [SerializeField] private TextMeshProUGUI messageText;
    [SerializeField] private float startTime = 60f;

    private int score;
    private int totalCollectibles;
    private int collectedCount;
    private float timeRemaining;
    private bool gameEnded;

    private void Start()
    {
        totalCollectibles = FindObjectsOfType<Collectible>().Length;
        timeRemaining = startTime;
        UpdateScoreUI();
        UpdateTimeUI();
        if (messageText != null) messageText.text = "";
    }
}
```



Anexo: GameManager 2/3

```
private void Update()
{
    if (Input.GetKeyDown(KeyCode.R)) { RestartGame(); return; }
    if (gameEnded) return;

    timeRemaining -= Time.deltaTime;
    if (timeRemaining <= 0f)
    {
        timeRemaining = 0f;
        UpdateTimeUI();
        LoseGame();
        return;
    }
    UpdateTimeUI();
}

public void AddScore(int points)
{
    if (gameEnded) return;
    score += points;
    collectedCount++;
    UpdateScoreUI();
    if (collectedCount >= totalCollectibles) WinGame();
}
```




Anexo: GameManager 3/3

```
public void LoseGame()
{
    if (gameEnded) return;
    gameEnded = true;
    if (messageText != null) messageText.text = "GAME OVER - Press R";
}

private void WinGame()
{
    gameEnded = true;
    if (messageText != null) messageText.text = "YOU WIN! - Press R";
}

private void RestartGame()
{
    Scene currentScene = SceneManager.GetActiveScene();
    SceneManager.LoadScene(currentScene.name);
}

private void UpdateScoreUI() { if (scoreText != null) scoreText.text = "Score: " + score; }
private void UpdateTimeUI() { if (timeText != null) timeText.text = "Time: " + Mathf.CeilToInt(timeRemaining); }
}
```




Siguiente paso

Variantes del prototipo

- Añadir niveles con más obstáculos.
- Crear boosters temporales.
- Mejorar feedback visual y sonoro.
- Usar ChatGPT como revisor antes de entregar.